

这篇文章将要深入理解HTML、URL和JavaScript的规范细则和解析器，以及在解析一段XSS脚本时他们之间有着怎样的差别。这些内容对读者的难易程度取决于读者对HTML规范和浏览器解析的知识是否充足。当然，我向您保证这篇文章比较长，因此请准备一小时或两小时来从中获益。在主题开始之前，请花费一点时间来看看下列语句并尝试回答：这些脚本能够正确执行吗？

基础部分

1	
2	URL 编码 "javascript:alert(1)"
3	<a href="javascript:%61%6c%65%72%74%28%32%29'
4	HTML字符实体编码 "javascript" 和 URL 编码 "alert(2)"
5	
6	URL 编码 ":"
7	
8	HTML字符实体编码 < 和 >
9	<textarea><script>alert(5)</script></textarea>
10	HTML字符实体编码 < 和 >
11	<textarea><script>alert(6)</script></textarea>

高级部分

1	<button onclick="confirm('7';)">Button</button>
2	HTML字符实体编码 " ' " （单引号）
3	<button onclick="confirm('8\u0027;)">Button</button>
4	Unicode编码 " ' " （单引号）
5	<script>alert(9);</script>
6	HTML字符实体编码 alert(9);
7	<script>\u0061\u006c\u0065\u0072\u0074(10);</script>
8	Unicode 编码 alert
9	<script>\u0061\u006c\u0065\u0072\u0074\u0028\u0031\u0031\u0029</script>
10	Unicode 编码 alert(11)
11	<script>\u0061\u006c\u0065\u0072\u0074(\u0031\u0032)</script>
12	Unicode 编码 alert 和 12
13	<script>alert('13\u0027)</script>
14	Unicode 编码 " ' " （单引号）
15	<script>alert('14\u000a')</script>
16	Unicode 编码换行符 (0x0A)

额外赠送

1	<pre></pre>
---	--

这些问题的答案在这里，测试页面在这里。如果你能答对大部分问题并且没有感觉困惑，看到这就停下吧。如果没有，那么余下的内容将能够很好的帮助你理解浏览器解析过程。

在解析一篇HTML文档时主要有三个处理过程：HTML解析，URL解析和JavaScript解析。每个解析器负责解码和解析HTML文档中它所对应的部分，其工作原理已经在相应的解析器规范中明确写明。

0x01 HTML解析

从XSS的角度来说，我们感兴趣的是HTML文档是如何被词法解析的，因为我们并不想让用户提供的数据最终被解析为一段可执行脚本的script标签。HTML词法解析细则在这里。HTML词法解析细则是一篇冗长的文档，这篇博文并不会覆盖它的所有内容。这篇博文只会覆盖有关文档解码如何结束，以及新token何时被创建这两个有趣的部分。

一个HTML解析器作为一个状态机，它从输入流中获取字符并按照转换规则转换到另一种状态。在解析过程中，任何时候它只要遇到一个'<'符号（后面没有跟'/'符号）就会进入“标签开始状态(Tag open state)”。然后转变到“标签名状态(Tag name state)”，“前属性名状态(before attribute name state)”.....最后进入“数据状态(Data state)”并释放当前标签的token。当解析器处于“数据状态(Data state)”时，它会继续解析，每当发现一个完整的标签，就会释放出一个token。

（译者注：词法解析是《编译原理》所涉及的内容，学习过编译原理的读者可以更好的理解“状态机”的工作原理）。

这里有三种情况可以容纳字符实体，“数据状态中的字符引用”，“RCDATA状态中的字符引用”和“属性值状态中的字符引用”。在这些状态中HTML字符实体将会从“&#...”形式解码，对应的解码字符会被放入数据缓冲区中。例如，在问题4中，“<”和“>”字符被编码为“<”和“>”。当解析器解析完“<div>”并处于“数据状态”时，这两个字符将会被解析。当解析器遇到“&”字符，它会知道这是“数据状态的字符引用”，因此会消耗一个字符引用（例如“<”）并释放出对应字符的token。在这个例子中，对应字符指的是“<”和“>”。读者可能会想：这是不是意味着“<”和“>”的token将会被理解为标签的开始和结束，然后其中的脚本会被执行？答案是脚本并不会被执行。原因是解析器在解析这个字符引用后不会转换到“标签开始状态”。正因为如此，就不会建立新标签。因此，我们能够利用字符实体编码这个行为来转义用户输入的数据从而确保用户输入的数据只能被解析成“数据”。

（译者注：这里要解释几个概念）

字符实体(character entities)

字符实体是一个转义序列，它定义了一般无法在文本内容中输入的单个字符或符号。一个字符实体以一个&符号开始，后面跟着一个预定义的实体的名称，或是一个#符号以及字符的十进制数字。

HTML字符实体(HTML character entities)

在HTML中，某些字符是预留的。例如在HTML中不能使用“<”或“>”，这是因为浏览器可能误认为它们是标签的开始或结束。如果希望正确地显示预留字符，就需要在HTML中使用对应的字符实体。

一个HTML字符实体描述如下：

字符显示	描述	实体名称	实体编号
<	小于号	<	360安全播报 (blog.360.cn)

需要注意的是，某些字符没有实体名称，但可以有实体编号。

字符引用 (character references)

字符引用包括“字符值引用”和“字符实体引用”。在上述HTML例子中，'<'对应的字符值引用为'<'，对应的字符实体引用为'<'。字符实体引用也被叫做“实体引用”或“实体”。)

现在你大概会明白为什么我们要转义“<”、“>”、“” (单引号)和“” (双引号)字符了。但为什么我们还要转义“&”呢？大概“&”是无辜的，任何跟在“&”后面的内容仅会被解释为字符引用，这并不会开始或闭合一个标签。事实上，“&”字符并不会打断HTML级别的转义过程，但它可能会打断其他级别的转义过程。我们将在JavaScript解析的部分讨论这个问题。

这里要提一下RCDATA的概念。要了解什么是RCDATA，我们先要了解另一个概念。在HTML中有五类元素：

1. 空元素(Void elements)，如<area>,
,<base>等等
2. 原始文本元素(Raw text elements)，有<script>和<style>
3. RCDATA元素(RCDATA elements)，有<textarea>和<title>
4. 外部元素(Foreign elements)，例如MathML命名空间或者SVG命名空间的元素
5. 基本元素(Normal elements)，即除了以上4种元素以外的元素

五类元素的区别如下：

1. 空元素，不能容纳任何内容（因为它们没有闭合标签，没有内容能够放在开始标签和闭合标签中间）。
2. 原始文本元素，可以容纳文本。
3. RCDATA元素，可以容纳文本和字符引用。
4. 外部元素，可以容纳文本、字符引用、CDATA段、其他元素和注释
5. 基本元素，可以容纳文本、字符引用、其他元素和注释

如果我们回头看HTML解析器的规则，其中有一种可以容纳字符引用的情况是“RCDATA状态中的字符引用”。这意味着在<textarea>和<title>标签中的字符引用会被HTML解析器解码。这里要再提醒一次，在解析这些字符引用的过程中不会进入“标签开始状态”。这样就可以解释问题5了。另外，对RCDATA有个特殊的情况。在浏览器解析RCDATA元素的过程中，解析器会进入“RCDATA状态”。在这个状态中，如果遇到“<”字符，它会转换到“RCDATA小于号状态”。如果“<”字符后没有紧跟着“/”和对应的标签名，解析器会转换回“RCDATA状态”。这意味着在RCDATA元素标签的内容中（例如<textarea>或<title>的内容中），唯一能够被解析器认做是标签的就是“</textarea>”或者“</title>”。当然，这要看开始标签是哪一个。因此，在“<textarea>”和“<title>”的内容中不会创建标签，就不会有脚本能够执行。这也就解释了为什么问题6中的脚本不会被执行。

我们来迅速看一下CDATA元素。任何在CDATA元素中的内容将不会触发解析器创建开始标签。闭合CDATA元素的标志是“]]>”序列。因此如果用户想逃出CDATA元素，就要用未经任何编码的“]]>”序列，不然是不会逃出CDATA元素的。

0x02 URL解析

URL解析器也是一个状态机模型，从输入流中进来的字符可以引导URL解析器转换到不同的状态。解析器的解析细则在[这里](#)。其中有很多有关安全或XSS转义的内容。

首先，URL资源类型必须是ASCII字母（U+0041-U+005A || U+0061-U+007A），不然就会进入“无类型”状态。例如，你不能对协议类型进行任何的编码操作，不然URL解析器会认为它无类型。这就是为什么问题1中的代码不能被执行。因为URL中被编码的“javascript”没有被解码，因此不会被URL解析器识别。该原则对协议后面的“:”（冒号）同样适用，即问题3也得到解答。然而，你可能会想到：为什么问题2中的脚本被执行了呢？如果你记得我们在HTML解析部分讨论的内容的话，是否还记得有一个情况叫做“属性值中的字符引用”，在这个情况中字符引用会被解码。我们将稍后讨论解析顺序，但在这里，HTML解析器解析了文档，创建了标签token，并且对href属性里的字符实体进行了解码。然后，当HTML解析器工作完成后，URL解析器开始解析href属性值里的链接。在这时，“javascript”协议已经被解码，它能够被URL解析器正确识别。然后URL解析器继续解析链接剩下的部分。由于是“javascript”协议，JavaScript解析器开始工作并执行这段代码，这就是为什么问题2中的代码能够被执行。

其次，URL编码过程使用UTF-8编码类型来编码每一个字符。如果你尝试着将URL链接做了其他编码类型的编码，URL解析器就可能不会正确识别。

0x03 JavaScript 解析

JavaScript解析过程与HTML解析过程有点不一样。JavaScript语言是一门内容无关语言。对应着有一份内容无关的语法来描述它。我们可以利用内容无关语法来解释JavaScript是如何解析的。ECMAScript-262细则在[这里](#)，语法文件在[这里](#)。

这里有一些与安全相关的事情：字符是如何被解码的？对一些字符进行转义是否有效？

开始之前，让我们回到HTML解析过程中的“原始文本”元素。我故意将HTML中的一部分留到这个章节是因为它与JavaScript解析有关。所有的“script”块都属于“原始文本”元素。“script”块有个有趣的属性：在块中的字符引用并不会被解析和解码。如果你去看“脚本数据状态”的状态转换规则，就会发现没有任何规则能转移到字符引用状态。这意味着什么？这意味着问题9中的脚本并不会执行。所以如果攻击者尝试着将输入数据编码成字符实体并将其放在script块中，它将不会被执行。

那像“\uXXXX”（例如\u0000,\u000A）这样的字符呢，JavaScript会解析这些字符来执行吗？简单的说：视情况而定。具体的说就是要看被编码的序列到底是哪部分。首先，像\uXXXX一样的字符被称作Unicode转义序列。从上下文来看，你可以将转义序列放在3个部分：字符串中，标识符名称中和控制字符中。

字符串中：当Unicode转义序列存在于字符串中时，它只会被解释为正规字符，而不是单引号，双引号或者换行符这些能够打破字符串上下文的字符。这项内容清楚地写在ECMAScript中。因此，Unicode转义序列将永远不会破坏字符串上下文，因为它们只能被解释成字符串常量。

ECMA-262 5.1版 6章 6节

“ECMAScript 与 JAVA 编程语言在对待Unicode转义序列时的行为不同。在Java程序中，如果Unicode转义序列\u000A出现在单行字符串注释中，它会被解释为行结束符（换行符），因此会导致接下来的Unicode字符不是注释的一部分。同样的，如果Unicode转义序列\u000A出现在Java程序的字符串常量中，它同样会被解释为行结束符（换行符），这在字符串常量中是不被允许的——如果需要在字符串常量中表示换行，需要用\n来代替\u000A。在ECMAScript程序中，出现在注释中的Unicode转义序列永远不会被解释，因此不会导致注释换行问题。同样地，ECMAScript程序中，在字符串常量中出现的Unicode转义序列会被当作字符串常量中的一个Unicode字符，并且不会被解释成有可能结束字符串常量的换行符或者引号。”

标识符名称中：当Unicode转义序列出现在标识符名称中时，它会被解码并解释为标识符名称的一部分，例如函数名，属性名等等。这可以用来解释问题10。如果我们深入研究JavaScript细则，可以看到如下内容：

“Unicode转义序列（如\u000A\u000B）同样被允许用在标识符名称中，被当作名称中的一个字符。而将'\'符号前置在Unicode转义序列串（如\u000A000B000C）并不能作为标识符名称中的字符。将Unicode转义序列串放在标识符名称中是非法的。”

控制字符:当用Unicode转义序列来表示一个控制字符时，例如单引号、双引号、圆括号等等，它们将不会被解释成控制字符，而仅仅被解码并解析为标识符名称或者字符串常量。如果你去看ECMAScript的语法，就会发现没有一处会用Unicode转义序列来当作控制字符。例如，如果解析器正在解析一个函数调用语句，圆括号部分必须为“(和)””，而不能是\u0028和\u0029。

总的来说，Unicode转义序列只有在标识符名称里不被当作字符串，也只有在标识符名称里的编码字符能够被正常的解析。如果我们回看问题11，它并不会被执行。因为“(11)”不会被正确的解析，而“alert(11)”也不是一个有效的标识符名称。问题12不会被正确执行要么是因为'\u0031\u0032'不会被解释为字符串常量（因为它们没有用引号闭合）要么是因为它们是ASCII型数字。问题13不会执行的原因是'\u0027'仅仅会被解释成单引号文本，而此时字符串是未闭合的。问题14能够执行的原因是'\u000a'会被解释成换行符文本，这并不会导致真正的换行从而引发JavaScript语法错误。

0x04 解析流

在讨论过HTML，URL和JavaScript解析之后，读者应该能够对“什么会被解码”、“在什么地方被解码”和“如何被解码”这几件事有了清楚的认识。现在，另一个重要的概念是所有这些是如何协同工作的？在网页中有很多地方需要多个解析器来协同工作。因此，对于解码和转义问题，我们将简要的讨论浏览器如何解析一篇文档。

当浏览器从网络堆栈中获得一段内容后，触发HTML解析器来对这篇文档进行词法解析。在这一步中字符引用被解码。在词法解析完成后，DOM树就被创建好了，JavaScript解析器会介入来对内联脚本进行解析。在这一步中Unicode转义序列和Hex转义序列被解码。同时，如果浏览器遇到需要URL的上下文，URL解析器也会介入来解码URL内容。在这一步中URL解码操作被完成。由于URL位置不同，URL解析器可能会在JavaScript解析器之前或之后进行解析。考虑如下两种情况

1	Example A:
2	Example B:

在例A中，HTML解析器将首先开始工作，并对UserInput中的字符引用进行解码。然后URL解析器开始对href值进行URL解码。最后，如果URL资源类型是JavaScript，那么JavaScript解析器会进行Unicode转义序列和Hex转义序列的解码。再之后，解码的脚本会被执行。因此，这里涉及三轮解码，顺序是HTML，URL和JavaScript。

在例B中，HTML解析器首先工作。然而接下来，JavaScript解析器开始解析在onclick事件处理器中的值。这是因为在onclick事件处理器中是script的上下文。当这段JavaScript被解析并被执行的时候，它执行的是“window.open()”操作，其中的参数是URL的上下文。在此时，URL解析器开始对UserInput进行URL解码并把结果回传给JavaScript引擎。因此这里一共涉及三轮解码，顺序是HTML，JavaScript和URL。

有没有可能解码次数超过3轮呢？考虑一下这个例子

1	Example C:
---	---

例C与例A很像，但不同的是在UserInput前多了window.open()操作。因此，对UserInput多了一次额外的URL解码操作。总的来说，四轮解码操作被完成，顺序是HTML，URL，JavaScript和URL。

此时此刻，读者应该已经获得解答博文开始提到的所有问题的必要知识。如果你有任何的问题，欢迎留言讨论。

0x05 总结

简而言之，作为攻击者为了弄明白如何让XSS向量逃逸出上下文，或者为了使你的应用能够正确编码用户的输入，你必须真正明白浏览器的解析原理以及它们（HTML，URL和JavaScript解析器）是如何协同工作的。只有这样，你才能从浏览器的角度去正确编码你的向量。